# End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering - Mini-Project Report
iMTech (CSE), Term I 2026-27 | Faculty: Prof. B. Thangaraju

| | |
|---|---|
| **Student** | Harjot Singh **Roll No.:** IMT2023064 |
| **GitHub repository** | https://github.com/harjotsingh-debug/Calculator-REST-API |
| **Docker Hub image** | https://hub.docker.com/r/harjotsingh12312/calculator-api |
| **Stack** | Java 21, Spring Boot 3.5, Maven, JUnit 5 / MockMvc, Git/GitHub, Jenkins, Docker, Docker Compose, Nginx |

## 1. Overview

A Spring Boot Calculator REST API is wrapped in a fully automated pipeline. A push to the **main** branch on GitHub triggers Jenkins, which compiles the code, runs the JUnit suite, builds a Docker image tagged with the build number, pushes it to Docker Hub, redeploys the Docker Compose stack and emails the result. No manual step is needed between *git push* and a healthy deployment. This was demonstrated by build #7, which Jenkins started on its own about 25 seconds after a merge was pushed and which ended with the new version live behind the proxy.

## 2. Architecture

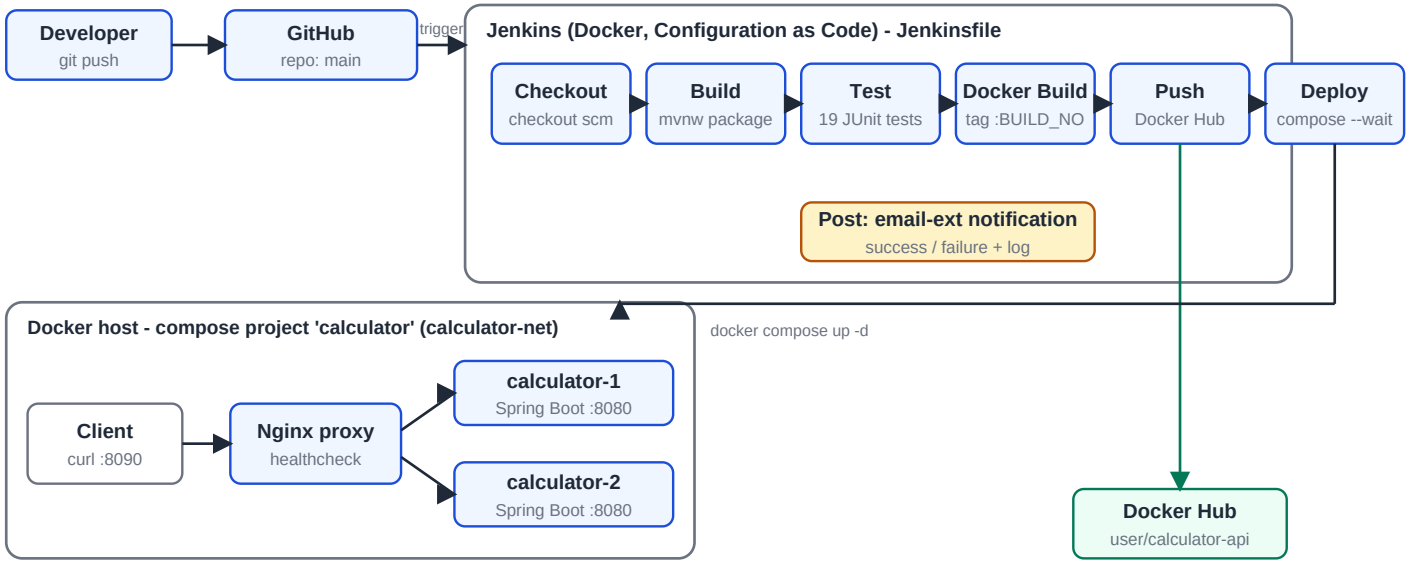



Figure 1 - Pipeline stages (top) and the deployed runtime (bottom left).

## 3. Application and Tests

| Endpoint | Behaviour |
|---|---|
| **GET /api/v1/add \| subtract \| multiply \| divide ?a=&b=** | JSON result, e.g. {"operation":"add","a":2,"b":3,"result":5} |
| **Errors (HTTP 400)** | Division by zero, missing parameter, non-numeric input. Body: {timestamp, status, error, message, path} from a @RestControllerAdvice |
| **GET /actuator/health, /actuator/info** | Spring Actuator probe used by Docker / Compose health checks; app version info |

- Arithmetic uses **BigDecimal**, so 0.1 + 0.2 returns exactly 0.3. Division is rounded to 16 significant digits.
- **11 JUnit 5 unit tests** (CalculatorServiceTest) cover every operation, negatives, decimals and divide-by-zero (including 0.0).
- **8 MockMvc integration tests** boot the full Spring context and hit the real endpoints, covering success, the three 400 cases and the health probe. All 19 run with *mvn test* and produce Surefire JUnit XML.

## 4. Pipeline Stages (Jenkinsfile)

| Stage | What it does |
|---|---|
| **Trigger** | githubPush() for a webhook, plus pollSCM('H/2 * * * *') because Jenkins runs on a laptop that GitHub cannot reach. disableConcurrentBuilds() prevents racing deploys. |
| **Checkout** | checkout scm fetches the exact commit that triggered the build. |
| **Build** | ./mvnw -B clean package -DskipTests. The Maven wrapper pins the Maven version, so any agent builds identically. |
| **Test** | ./mvnw -B test. The junit step publishes target/surefire-reports/*.xml (Test Result Trend). A failing test exits non-zero and fails the build before an image is produced. |
| **Docker Build** | docker build -t <user>/calculator-api:$BUILD_NUMBER -t ...:latest, plus docker compose build proxy for the Nginx image. |
| **Push** | withCredentials(usernamePassword 'dockerhub-creds'). The token comes from the Jenkins credentials store and is passed with --password-stdin, so it never appears in the log or the repo. Pushes both tags. |
| **Deploy** | docker compose up -d --no-build --remove-orphans --wait. This is idempotent: only changed containers are recreated, and --wait fails the stage unless every service is healthy. A smoke test then calls /actuator/health and /api/v1/add through the proxy. |
| **Post** | emailext sends a success or failure mail (failure attaches the console log) via Gmail SMTP with an App Password stored as a credential, then prunes dangling images. |

## 5. Containerisation and Deployment

- **Multi-stage Dockerfile:** maven:3.9.11-eclipse-temurin-21 builds the JAR, and the runtime is eclipse-temurin:21-jre-alpine with only the JAR copied in. pom.xml is copied and dependency:go-offline runs before src/, so the dependency layer stays cached. The image runs as a non-root *app* user with an explicit EXPOSE 8080 and a HEALTHCHECK, and a .dockerignore keeps .git, target/ and secrets out of the build context. A single *docker run -p 8080:8080* serves the API.
- **docker-compose.yml:** two *calculator* replicas behind an *Nginx* reverse proxy on the bridge network calculator-net. Both services have health checks (/actuator/health), and the proxy starts only after the API is service_healthy. Restart policy is unless-stopped, and environment variables set SERVER_PORT, SPRING_PROFILES_ACTIVE, JAVA_TOOL_OPTIONS and the image tag. Only the proxy publishes a host port (8090). Nginx resolves the replicas through Docker DNS (127.0.0.11) at request time, so redeployed containers are picked up.
- **Jenkins as code:** jenkins/ holds a Jenkins image with the Docker CLI and Compose, the plugins, and a Configuration-as-Code file. That file creates the admin user, credentials, global variables, SMTP settings and the pipeline job, with secrets read from a git-ignored .env file.

## 6. Version Control Practices

The history has 17 commits, built up incrementally from scaffold to service, API, Docker, Compose, Jenkins and docs. Four feature branches (feature/docker, feature/jenkins-pipeline, feature/jenkins-docker, feature/actuator-info) were merged into main with --no-ff merge commits. The .gitignore covers target/, IDE files, .env and key/credential files. No credentials exist in the repository, and all secrets live in the Jenkins credentials store.

## 7. Challenges Faced

- **Deploying from a containerised Jenkins:** Compose resolves bind-mount paths on the Docker host, not inside the Jenkins container, so mounting nginx.conf from the workspace failed. The config is now baked into a small proxy image built by the pipeline.
- **Docker socket permissions:** the jenkins user could not use /var/run/docker.sock (root:root, 660). This was solved with group_add: 0 in Jenkins' compose file.
- **Proxy health check unhealthy:** inside Alpine, *localhost* resolved to IPv6 ::1 while Nginx listened on IPv4 only. Probing 127.0.0.1 fixed it.
- **Failure e-mail crashed:** email-ext's attachLog needs the matrix-project plugin, which was missing. A test run that failed on purpose exposed this, and the plugin was added to plugins.txt.

- **Early red builds (#3-#5 in the Stage View):** Jenkins was first validated against a local checkout with placeholder Docker Hub credentials, so Push failed with *unauthorized* and Deploy was skipped, as intended. Builds #1-#2 failed at checkout because of git's safe.directory ownership check on the mounted repo. After the real token was configured, every build from #6 onwards passed end to end.
- **Host port conflicts:** 8080 and 8081 were already used by other local services, so Jenkins uses 8085 and the proxy port is configurable (PROXY_PORT, default 8090).
- **Number formatting:** after removing trailing zeros, BigDecimal 10 serialised as 1E+1. Jackson's write-bigdecimal-as-plain fixed this.

## Appendix - Screenshots and Evidence

### A1 - Console highlights of build #7 (auto-triggered by a push)

```
Started by an SCM change
Checking out refs/remotes/origin/main
[Test]   Tests run: 19, Failures: 0, Errors: 0, Skipped: 0 - BUILD SUCCESS
[Push]   7: digest: sha256:fb29ba45b9fa...  latest: digest: sha256:fb29ba45b9fa...
[Deploy] calculator-calculator-1, calculator-calculator-2, calculator-proxy-1: Healthy
[Deploy] {"status":"UP"}   {"operation":"add","a":2,"b":3,"result":5}
[Post]   Sending email to: jogamerzne658@gmail.com
Finished: SUCCESS
```

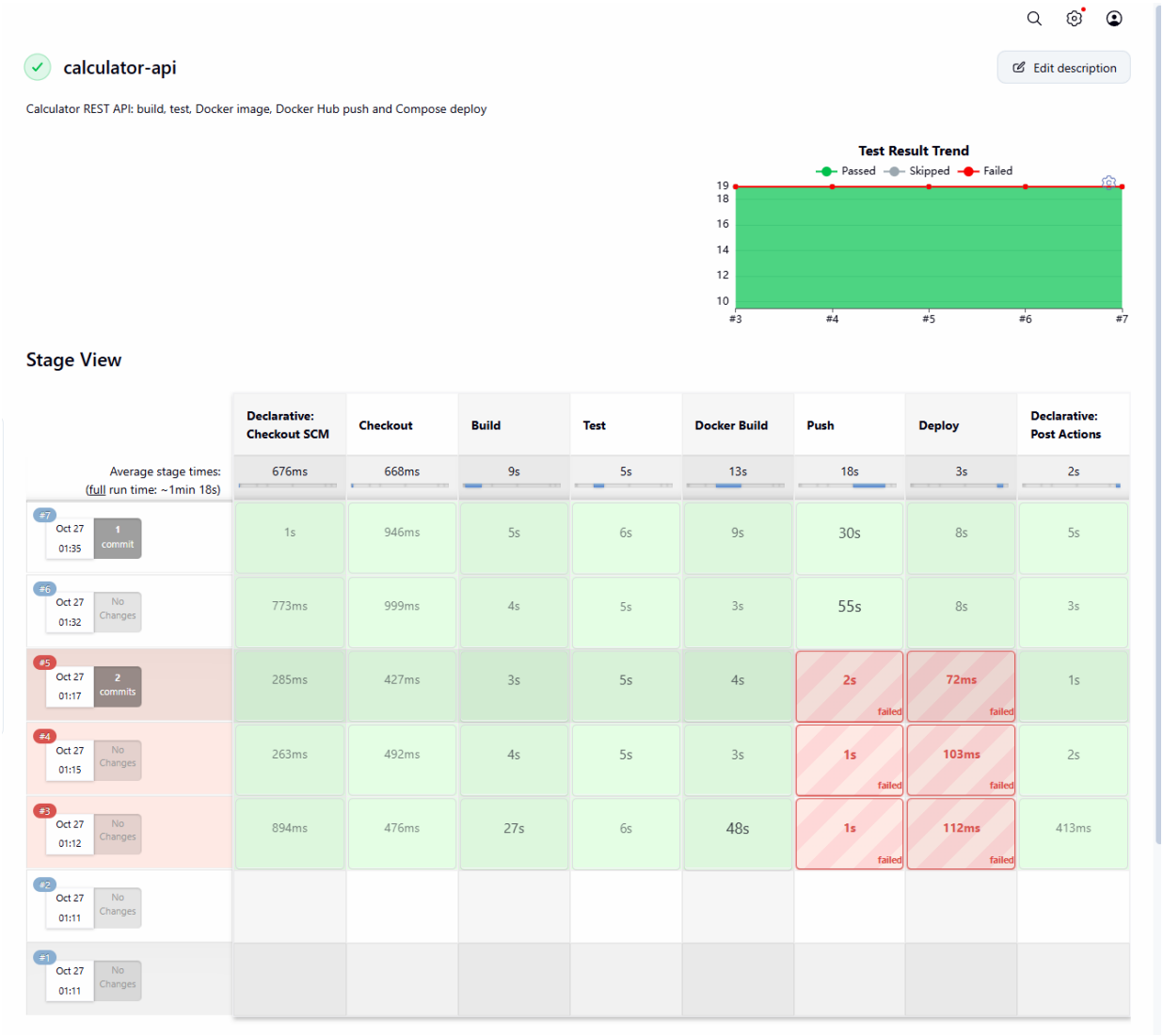
### A2 - Running containers after deployment (docker ps)

```
PS C:\Users\Harjo> docker ps --filter name=calculator --filter name=jenkins
CONTAINER ID   IMAGE                              COMMAND                  CREATED          STATUS                    PORTS                                       NAMES
54a3bcc1f4ab   calculator-proxy:10                "/docker-entrypoint.…"   54 seconds ago   Up 47 seconds (healthy)   0.0.0.0:8090->80/tcp, [::]:8090->80/tcp     calculator-proxy-1
67a07aa7d66d   harjotsingh12312/calculator-api:10 "java -XX:MaxRAMPerc…"   54 seconds ago   Up 53 seconds (healthy)   8080/tcp                                    calculator-calculator-2
1efcc25b6e4c   harjotsingh12312/calculator-api:10 "java -XX:MaxRAMPerc…"   54 seconds ago   Up 53 seconds (healthy)   8080/tcp                                    calculator-calculator-1
2eb6812f4a0a   calculator-jenkins:latest          "/usr/bin/tini -- /u…"   26 minutes ago   Up 26 minutes             0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp jenkins-jenkins-1
PS C:\Users\Harjo>
```

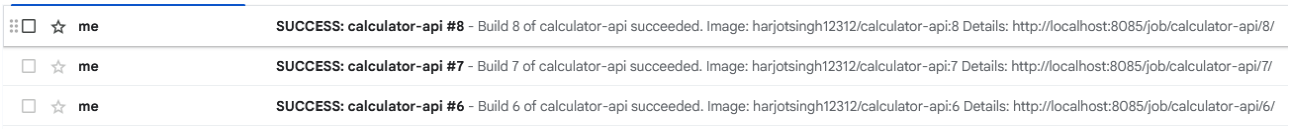
### A3 - Sample API responses through the Nginx proxy

```
PS C:\Users\Harjo> curl.exe "http://localhost:8090/api/v1/add?a=2&b=3"
{"operation":"add","a":2,"b":3,"result":5}
PS C:\Users\Harjo> curl.exe "http://localhost:8090/api/v1/subtract?a=10&b=4"
{"operation":"subtract","a":10,"b":4,"result":6}
PS C:\Users\Harjo> curl.exe "http://localhost:8090/api/v1/multiply?a=6&b=7"
{"operation":"multiply","a":6,"b":7,"result":42}
PS C:\Users\Harjo> curl.exe "http://localhost:8090/api/v1/divide?a=10&b=4"
{"operation":"divide","a":10,"b":4,"result":2.5}
PS C:\Users\Harjo> curl.exe -i "http://localhost:8090/api/v1/divide?a=1&b=0"
HTTP/1.1 400
Server: nginx/1.27.5
Date: Mon, 26 Oct 2026 20:29:17 GMT
Content-Type: application/json
Transfer-Encoding: chunked
Connection: keep-alive

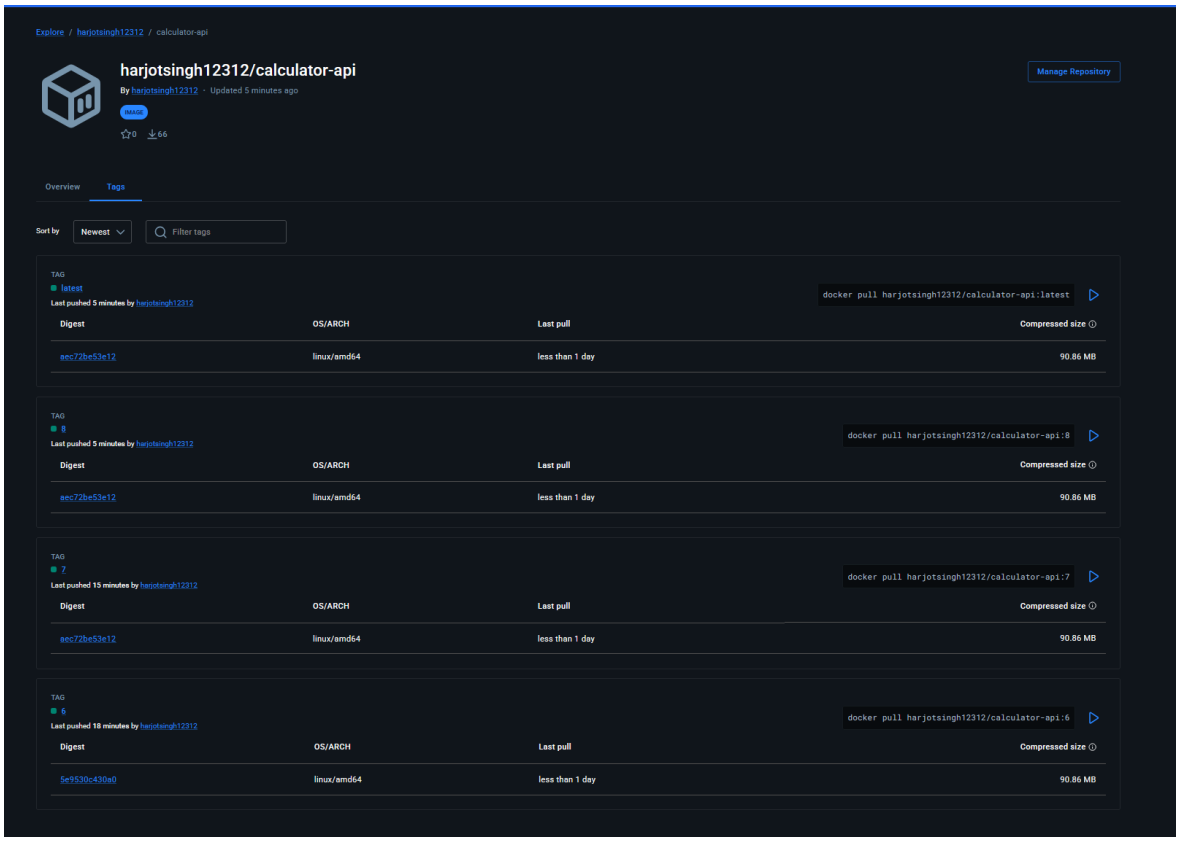
{"timestamp":"2026-10-26T20:29:17.106625083Z","status":400,"error":"Bad Request","message":"Division by zero is not allowed","path":"/api/v1/divide"}
PS C:\Users\Harjo> curl.exe "http://localhost:8090/api/v1/add?a=abc&b=1"
{"timestamp":"2026-10-26T20:29:25.136309878Z","status":400,"error":"Bad Request","message":"Parameter 'a' must be a valid number but was 'abc'","path":"/api/v1/add"}
PS C:\Users\Harjo> curl.exe "http://localhost:8090/actuator/health"
{"status":"UP","groups":["liveness","readiness"]}
PS C:\Users\Harjo>
```




A4 - Jenkins job: Test Result Trend (19 passing) and Stage View. Builds #6 and #7 are green across all stages, and #7 was triggered automatically by a push ("1 commit").




A5 - Success e-mails for builds #6, #7 and #8 (email-ext via Gmail SMTP)




A6 - Public Docker Hub repository with build-number tags 6, 7, 8 and latest. Tags 7, 8 and latest share one digest: build #8 changed only docs/, which .dockerignore excludes, so every layer came from cache and the image was identical.